PART A — FUNDAMENTALS

1. What is Injection?

- It is a security vulnerability where untrusted input is interpreted as a part of command or query.
- It may allow attackers to execute unintended instructions.

2. What is untrusted input?

It is a data received from a source where it can't be trusted. It must be validated before any application process it.

3. Give five examples of user-controlled input.

- Search boxes
- Login forms
- File uploads
- Contact forms
- URL parameters

4. What is an interpreter?

It is a program that reads and executes instruction written in programming language.
Eg: SQL database

5. Explain the difference between Data and Instructions.

- Data: Information provided to or processed by a system.
- Instructions: Commands that tells a system what actions to perform.

6. What is SQL Injection at a conceptual level?

SQL injection occurs when an attacker manipulate input that is part of SQL query. Leading access to unauthorized access, modification or deletion of data.

7. What is OS Command Injection at a conceptual level?

OS Command injection it occurs when untrusted input is understood as part of OS command. Which may allow attackers to execute commands on the system.

8. Why is raw string concatenation with untrusted input risky?

It combines user-controlled input directly with commands. Attackers may manipulate the input so it changes the intended instructions and cause injection attacks.

PART B — DEFENSIVE CONTROLS

9. What is a parameterized query?

- It is a SQL query that uses placeholders for user-provided values.
- It keeps user instruction separate from SQL instructions.

10. What is a prepared statement?

It is a SQL statement prepared by database with placeholders for values. The values are separated when statement is executed.

11. Why do parameterized queries improve security?
The separate SQL commands from user input preventing user input from interpreted as executable SQL instruction and reducing SQL injection risks.

12. What is input validation?
It is the process of checking user-provided data before processing it also ensuring that the input follows expected format, type and rules.

13. What is allowlist validation?
It defines which input are acceptable. Only values that match the predefined rules are accepted, while unexpected input are rejected.

14. Why is client-side validation alone not sufficient?
Client-side validation can be modified by attackers. Attackers can send request directly to server without using the website interface.

15. Why should important validation also happen server-side?

- It is a validation ensures that input is checked by the trusted backend.
- It prevents malicious data from being processed even if browser side checks are bypassed

16. Explain Least Privilege for a database account.

- A database account should only receive the minimum permissions required to perform its tasks.
- It should not have unnecessary privileges including administrator access reducing potential damage even if compromised.

17. Why should detailed database errors not normally be displayed publicly?
Detailed errors may reveal database names, table structures, queries, or internal system information.

PART C — SAFE PRACTICAL ANALYSIS

Fictional application:

Student Search Portal

Input: Student ID

Current Design:

- Browser performs basic validation
- Backend trusts the value
- Raw input is concatenated into a database query
- Database account has administrator privileges
- Detailed database errors are shown publicly

Answer:

1. Identify every security weakness.

- Client-side validation only
- Raw SQL query concatenation
- Database administrator privilege
- Database detailed error shown previously
- Backend trusts user input

2. Which OWASP Top 10:2025 category primarily applies: A:05:2025 - Injection

3. Why is client-side validation insufficient?

Client-side validation can be modified by attacker. If attacker request directly instead of website it can potentially compromise.

4. What should replace raw query concatenation?

- Parametrized queries or prepared statements.
- They separate SQL instruction from user-provided input and help prevent SQL injection

5. What server-side validation could be applied to Student ID?

- Check that the Student ID follows the expected format.
- Check the expected length.
- Verify that the Student ID exists in the database.

6. How should database privileges be improved?

- Apply the Principle of Least Privilege.
- Remove unnecessary administrator privileges.

7. How should public error handling be improved?

- Display generic error messages to public users.
- Restrict access to internal error information.

8. Give a secure high-level workflow for processing Student ID.

- Receive the Student ID from the user.
- Perform server-side input validation.
- Reject invalid input.
- Use a parameterized SQL query.
- Execute the query using a database account with limited privileges.
- Check whether the user is authorized to access the requested record.
- Return only the permitted information.
- Log relevant errors securely without exposing internal details.

PART D — DESIGN COMPARISON

Design A:

Query structure is prepared separately.
Student ID is passed as a bound parameter.

Design B:

Raw Student ID is concatenated directly into query text.

Answer:

1. Which design is safer: Design A

2. Explain why.

- Design A separates SQL query from user-provided data.
- Student ID is passed as a bound parameter so clears it won't be interpreted as a SQL command.

3. Which security principle does the safer design follow?

- Separation of Instructions and Data.
- This shows user input is interpreted as data than executable SQL commands